

SaaS Bokningssystem

Project Overview & Technical Specification

Version: 1.0
Datum: 2026-01-07
Team: 3 utvecklare
Tidsram: 2.5 veckor

Executive Summary

Detta projekt syftar till att bygga en production-ready SaaS-applikation för organisationer och företag där användare kan boka tider eller resurser. Systemet är multi-tenant, säkert, skalbart och byggt enligt enterprise-praxis med ett tydligt avgränsat MVP-scope.

■ Multi-tenant arkitektur	■ Enterprise-grade säkerhet
■ Skalbar design	■■ Clean Architecture
■ CQRS pattern	■■ Cloud-native deployment

Projektmål

- Leverera en **MVP-version** av ett generiskt bokningssystem inom 2.5 veckor
- Implementera **multi-tenant** arkitektur med fullständig dataisolering
- Bygga säkert autentiserings- och auktoriseringssystem
- Skapa flexibelt bokningssystem som kan hantera olika typer av resurser
- Etablera CI/CD pipeline för continuous deployment
- Dokumentera arkitektur och API för framtida utveckling

Målgrupp

Primär målgrupp: Organisationer och företag (Admins)

Företag och organisationer som behöver hantera bokningar av resurser såsom mötesrum, utrustning, personal, eller tidsbokningar för tjänster.

Sekundär målgrupp: Slutanvändare (Customers / Members / Employees)

Anställda, medlemmar eller kunder som behöver boka tider eller resurser inom organisationen.

Arkitekturöversikt

Multi-tenant SaaS-applikation

Systemet är byggt som en **monolitisk men modulärt uppbyggd applikation** med multi-tenant-stöd. Detta innebär att flera organisationer delar samma applikationsinstans och databas, men data är strikt isolerat per tenant genom användning av TenantId.

Clean Architecture

Projektet följer Clean Architecture-principerna med tydlig separation av concerns:

- **Domain Layer** - Affärslogik och entities
- **Application Layer** - Use cases, CQRS handlers, interfaces
- **Infrastructure Layer** - Databas, external services, repositories
- **Presentation Layer** - API controllers, authentication, validation

CQRS Pattern

Vi använder **Command Query Responsibility Segregation** (CQRS) pragmatiskt med MediatR för att separera läs- och skrivoperationer. Detta ger oss bättre skalbarhet och enklare testning.

Tech Stack

Backend

Komponent	Teknologi	Syfte
.NET	8 eller 9	Runtime och framework
ASP.NET Core	Web API	REST API implementation
Entity Framework Core	Latest	ORM och dataåtkomst
PostgreSQL	15+	Primär databas
MediatR	Latest	CQRS implementation
Serilog	Latest	Structured logging
FluentValidation	Latest	Input validation
AutoMapper	Latest	Object mapping

Frontend

Val 1: Blazor Web App

- C#-baserad, tight integration med backend
- Server-side eller WebAssembly rendering
- Mindre JavaScript-kunskap krävs

Val 2: React + TypeScript

- Modern SPA-arkitektur
- Större ekosystem och flexibilitet
- Separation mellan frontend och backend

Rekommendation: React för denna sprint med tanke på teamets erfarenhet.

Infrastructure & DevOps

Komponent	Teknologi/Plattform
Cloud Hosting	Azure App Service
Database	Azure Database for PostgreSQL
Secrets Management	Azure Key Vault
Logging & Monitoring	Application Insights
CI/CD	GitHub Actions
Container Registry	Azure Container Registry (optional)

Säkerhet & Autentisering

Roller & Behörigheter

Roll	Beskrivning	Behörigheter
Owner	Skapare av organisation	Full kontroll över tenant, kan bjuda in admins
Admin	Administratör	Hantera resurser, bokningsregler, användare
User	Slutanvändare	Boka och avboka tider, se egna bokningar

Autentiseringsflöde

- **1. Registrering:** Owner skapar organisation (tenant) via registreringsformulär
- **2. Invitation:** Admin bjuder in users via e-post med invite-token
- **3. Aktivering:** User klickar på länk i e-post och sätter lösenord
- **4. Login:** E-post + lösenord authentication
- **5. Token:** JWT token utfärdas med claims (UserId, TenantId, Role)
- **6. MFA (Optional):** Multi-factor authentication för Admin-roller

Säkerhetsåtgärder

- HTTPS-only kommunikation
- Password hashing med BCrypt eller PBKDF2
- JWT tokens med kort expiration time (15-30 min)
- Refresh tokens för session management
- Rate limiting på authentication endpoints
- Tenant isolation via global query filters
- Input validation på alla endpoints
- SQL injection protection via EF Core
- Audit logging för kritiska operationer

Icke-funktionella Krav

Kategori	Krav	Implementation
Säkerhet	HTTPS, tenant isolation	SSL cert, query filters, JWT
Prestanda	Response time < 200ms	Indexering, caching, lazy loading
Skalbarhet	Hantera 100+ tenants	Stateless API, connection pooling
Tillgänglighet	99.5% uptime	Health checks, auto-restart, monitoring
Observability	Centralized logging	Serilog + Application Insights
Maintainability	Clean code, tests	Clean Architecture, >70% coverage
Compliance	GDPR-ready	Data export, deletion capabilities

Kvalitetssäkring

Testing Strategy

- **Unit Tests:** Domänlogik, validators, handlers (mål: 70% coverage)
- **Integration Tests:** API endpoints, authentication, tenant isolation
- **E2E Tests (Optional):** Critical user flows med Playwright
- **Performance Tests:** Load testing av booking creation endpoint
- **Security Tests:** OWASP top 10 vulnerabilities

Code Quality

- Code reviews för alla pull requests
- Automated linting (StyleCop, ESLint)
- SonarQube eller CodeQL för static analysis
- Branch protection på main/develop
- Conventional commits för changelog generation

Utanför MVP-scope

Följande funktioner är **explicit exkluderade** från MVP men kan ingå i framtida versioner:

- **Betalningar & Fakturering:** Ingen integration med betalningslösningar (Stripe, Swish, etc.)
- **BankID / Stark autentisering:** Endast e-post/lösenord i MVP
- **Avancerad CRM:** Ingen kundhantering, marknadsföring eller försäljningspipeline
- **Mobilappar:** Endast responsiv webbapplikation
- **Realtidsfunktioner:** Ingen WebSocket-baserad realtidsuppdatering
- **Externa integrationer:** Ingen kalendersync (Google/Outlook) eller externa API:er
- **Avancerad rapportering:** Endast grundläggande statistik och export
- **Multi-språkstöd:** Endast svenska i MVP
- **White-labeling:** Ingen anpassning per tenant av branding
- **Advanced scheduling:** Ingen recurring bookings eller complex scheduling rules

Definition of Done (MVP)

MVP anses färdigt när följande kriterier är uppfyllda:

- Organisation kan registreras och onboardas
- Admin kan konfigurera bokningsbara resurser och tider
- Users kan bjudas in och aktivera sina konton
- Users kan boka och avboka tider
- Tenant-data är fullständigt isolerad och testad
- API är dokumenterat med Swagger/OpenAPI
- Applikationen är deployad i Azure med CI/CD
- Health checks och logging är implementerat
- Enhetstester uppnår minst 70% code coverage
- Integration tests täcker kritiska flöden
- Performance test visar acceptabel response time
- Systemet är redo att byggas vidare på

Team & Kommunikation

Team Composition

Teamet består av **3 utvecklare** med följande ansvarsområden:

- **Developer 1 - Backend Lead:** API design, authentication, CQRS implementation
- **Developer 2 - Database & Infrastructure:** Data modeling, EF Core, Azure deployment
- **Developer 3 - Frontend & Integration:** React app, API integration, E2E testing

Arbetsmetodik

Vi arbetar med **Scrum-inspirerad agil utveckling** anpassad för kort tidsram:

- Daily standup (15 min) - Vad gjorde jag, vad ska jag göra, blockers
- Sprint planning - Måndag vecka 1, planera hela sprinten
- Code reviews - Alla PRs granskas av minst en annan utvecklare
- Demo - Fredag vecka 2, visa progress för stakeholders
- Retrospective - Sista dagen, vad gick bra/dåligt

Verktyg

- **GitHub:** Version control, project board, issues
- **Discord/Slack:** Daglig kommunikation
- **Notion/Confluence:** Dokumentation och kunskapsdelning
- **Figma (optional):** UI/UX design och mockups

Framgångskriterier

- **Teknisk kvalitet:** Clean Architecture implementerad, 70%+ test coverage
- **Funktionalitet:** Alla MVP-features är implementerade och testade
- **Säkerhet:** Inga kritiska sårbarheter, tenant isolation verifierad
- **Prestanda:** API response time < 200ms för 95% av requests
- **Deployment:** Applikationen är live i Azure med working CI/CD
- **Dokumentation:** API dokumenterad, arkitektur beskriven, setup guide skriven
- **Demobar:** Systemet kan demoas med realistic seed data
- **Skalbar kod:** Enkelt att lägga till nya features post-MVP

Resurser & Dokumentation

GitHub Repository: [INSERT URL]

Project Board: [INSERT URL]

API Documentation: [INSERT SWAGGER URL]

Deployment: [INSERT AZURE URL]

Team Communication: [INSERT DISCORD/SLACK]